

Day 4 — WiFi 上云与 MQTT (完整代码版 • F407VET6 新核心板版)

已按新核心板更新：ESP8266 为**外接**（新板无板载接口），USART3 PB10/PB11 + DMA 空闲中断；主线走华为云 IoTDA（工程 app_mqtt.c）

今日目标：MQTTX 订阅对应 topic 能持续收到 JSON 格式的传感器数据，断开 WiFi 后重连能自动恢复。

开发顺序：USART3+DMA 配置 → AT 指令封装 → WiFi 连接 → MQTT 连接 → JSON 封装发布 → 重连保活 → MqttTask 集成 → Web 仪表盘。

1. 第一步：ESP8266 接线（外接，USART3）

新板差异：F407VET6 新核心板**没有板载 ESP8266 接口**，模块外接，走 USART3。

ESP8266 引脚	STM32 引脚	说明
VCC	独立 3.3V（AMS1117 单独一路， $\geq 300\text{mA}$ ）	必须 3.3V ，5V 会烧；发射瞬态电流 300-500mA
GND	GND	共地
TXD	PB11 (USART3_RX)	ESP8266 TX → 单片机 RX
RXD	PB10 (USART3_TX)	ESP8266 RX → 单片机 TX
CH_PD / EN	3.3V（10k 上拉）	使能引脚，必须拉高
GPIO0	10k 上拉（或悬空）	正常运行模式
RST	悬空	复位（低有效），可选

供电警告：ESP8266 发射时峰值电流可达 300-500mA，直接从 STM32 的 3.3V LDO 取电可能导致电压跌落、模块重启。建议用独立 3.3V 稳压（AMS1117-3.3 单独一路）或 USB 转 3.3V 模块供电，GND 与 STM32 共地。

2. 第二步：USART3 + DMA 接收驱动

工程封装：`bsp_uart.c`（USART3：PB10=TX、PB11=RX，115200 8N1，DMA 循环接收 + IDLE 空闲中断判帧）。

```
/* CubeMX 关键配置 */
Connectivity → USART3 → Mode: Asynchronous
Baud Rate: 115200, Word Length: 8 Bits, Parity: None, Stop Bits: 1
DMA Settings → USART3_RX (Circular, Priority: High)
NVIC → 开启 USART3 全局中断
/* 引脚确认: PB10 = USART3_TX, PB11 = USART3_RX */
```

HAL_UARTEx_ReceiveTxdle_DMA: 结合 DMA 循环接收和串口空闲中断, 高效接收不定长数据 (AT 响应)。
若 HAL 库较老无此函数, 可用 HAL_UART_Receive_DMA() + 手动开启空闲中断
(`__HAL_UART_ENABLE_IT(&huart3, UART_IT_IDLE)`)。工程 `bsp_uart.c` 已封装好, 上层无需关心。

3. 第三步: AT 指令封装与 WiFi 连接

工程封装: `bsp_esp8266.c` (阻塞式 AT 指令驱动, 依赖 BSP_UART3)。

```
/* bsp_esp8266.h 核心 API */
void BSP_ESP8266_Init(void); /* 等待模块上电稳定 */
ESP8266_Status_t BSP_ESP8266_TestAT(uint32_t timeout_ms); /* AT */
ESP8266_Status_t BSP_ESP8266_SetMode(ESP8266_Mode_t mode); /* AT+CWMODE=1 STA */
ESP8266_Status_t BSP_ESP8266_JoinAP(const char *ssid, const char *pwd); /* AT+CWJAP, 等 WIFI GOT IP */
ESP8266_Status_t BSP_ESP8266_GetIP(char *ip, uint32_t size); /* AT+CIFSR */

/* 典型初始化序列 (工程 APP_Init / MqttTask 中调用) */
BSP_ESP8266_Init();
if (BSP_ESP8266_TestAT(1000) == ESP8266_OK) BSP_UART1_Printf("ESP8266 AT OK

");
BSP_ESP8266_SetMode(ESP8266_MODE_STA);
if (BSP_ESP8266_JoinAP("你的WiFi名称", "你的WiFi密码") == ESP8266_OK) {
    char ip[32]; BSP_ESP8266_GetIP(ip, sizeof(ip));
    BSP_UART1_Printf("WiFi connected, IP: %s

", ip);
}
```

验收: 串口打印 "WiFi connected, IP: 192.168.x.x"。AT 无响应先查 TX/RX 是否接反、波特率 115200、供电是否充足。

4. 第四步: MQTT 协议实现 (工程主线: 华为云 IoTDA)

工程采用 `app_mqtt.c`: ESP8266 透传 TCP + Paho MQTTClient-C 移植 (`mqtt_port.c`), 连接华为云 IoTDA, 实现属性上报与命令下行。

```

/* 华为云主题 (app_mqtt.c 中拼接 device_id) */
上报属性(发布): $oc/devices/{device_id}/sys/properties/report
接收命令(订阅): $oc/devices/{device_id}/sys/commands/#
命令响应(发布): $oc/devices/{device_id}/sys/commands/response/request_id={rid}

/* 上报 JSON (物模型 Sensor 服务):
* {"services":[{"service_id":"Sensor","properties":{
*   "sht_temperature":26.3,"sht_humidity":58.0,
*   "pitch":1.2,"roll":0.5,"ax":12,"ay":3,"az":1000,
*   "bus_voltage":24000,"current":1200,"power":28800,
*   "relay_state":1}}]}
* 命令下行: RELAY_Control / LED_Control (远程控制继电器/灯) */

```

调试用公共 broker 测试: 如改用 broker.emqx.io:1883, 直接 AT+MQTT 指令即可
 (AT+MQTTCONN=0,"broker.emqx.io",1883,1;
 AT+MQTTPUB=0,"motor/sensor/data",len,payload,0,0)。工程主线已定为华为云 IoTDA (配置指南与实验报告见工程目录)。

5. 第五步: MqttTask 集成 (2 秒周期发布)

```

/* app_mqtt.c 任务主循环逻辑 (节选) */
for (;;) {
  /* 1. 断线检测与重连: WiFi 断开 → JoinAP; TCP 断开 → 重连 */
  /* 2. 每 2s 从统一快照(APP_SENSOR)取数, 拼 JSON 上报属性 */
  /* 3. MQTT_Yield 处理下行命令回调 (继电器/灯控制 + 命令响应) */
  /* 4. 告警消息: AlarmTask 触发后经 MQTT 发布告警 topic */
  osDelay(2000);
}

```

Day 4 验收: 华为云设备详情页 / MQTTX 订阅对应 topic 能持续收到 JSON 格式传感器数据; 断开 WiFi 后自动重连并恢复上报。